

Internet scanning and service discovery

Follow-along: find the hosts, name the services, get in

Overview

Definition: Reconnaissance

The stage of an attack that runs before anything is exploited: finding out which addresses in a range hold live hosts, which of those hosts accept connections, and what software answers on each one. It produces a target list, not access.

Every attack on a network starts with a question the attacker cannot answer from outside: what is actually in there. An address range is public knowledge, but which of its 254 usable addresses hold a running host, and which of those hosts run software worth attacking, is not. **Reconnaissance** is the work of turning the range into that list, and it is almost entirely done with tools that send ordinary, well-formed packets and write down what comes back.

This lab gives you one /24 you do not otherwise know anything about and has you map it in three passes: find the live addresses, find what each one is running, and then use what you found to get a login on one of them. Each pass narrows the one before it. Only some of the range answers, only some of what answers is running anything, and one of the services that is running gives away enough about a second host to attack an account on it. Detecting a scan like this, and blocking it, is a separate lab; nothing here defends anything.

Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab # first time
cd ~/MiniLab && git pull # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

If it exits with a Docker error, follow the setup in the repository's `README.md` and run it again.

The root menu lists one lab per row. Select **Internet scanning and service discovery** to open its actions:

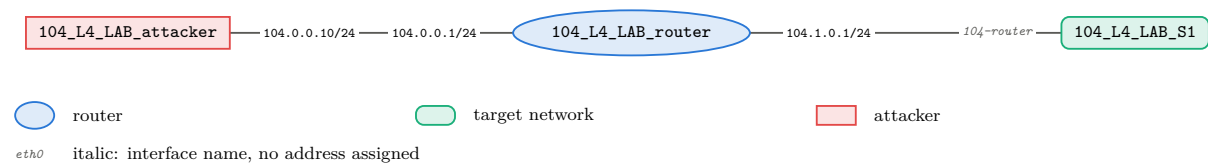
- **Spawn** creates this lab's containers and wiring: the machine you work from, the router, and the network you are asked to survey. Wait for the panel to turn green before you go on. The first spawn of a lab also builds its images, which takes a few minutes; later spawns take seconds.
- **Status** prints the addressing you have been given and confirms each service in the lab is running and reachable. It deliberately does not tell you which addresses those services sit on, because working that out is the exercise. Status takes over the whole screen: the arrow keys scroll its output, and Enter on **Done** closes it.

- **Shell command for a node** asks which node and prints the `docker exec -it ...` line to paste into your own terminal. Every step below runs on **attacker**; the other nodes are listed so you can look at a host after you have found it, not before.
- **Reset to baseline** deletes the scan output you have accumulated and restarts the services, so you can run a part again from nothing.
- **Teardown** removes the lab.
- **Open handout** opens this document.

Spawn the lab now, then work through Part 1 in your own terminal. **Back** (or Esc) returns to the lab list.

The setup

You work from one machine, on the far side of a router from the network you are surveying. That is the whole point: you are outside, and the only thing you have been told is the address range.



Nothing is drawn inside the target network because nothing about it has been given to you. What you do have:

What	Value	Container
your machine	104.0.0.10	104_L4_LAB_attacker
your interface	104-ext	
your gateway	104.0.0.1	104_L4_LAB_router
the target range	104.1.0.0/24	

The router forwards between your subnet and the target range and filters nothing, so every probe you send arrives and every reply comes back. The addresses inside 104.1.0.0/24, how many hosts hold them, and what each one runs are what Parts 1 and 2 have you establish.

Your machine carries stock tooling; there is no bespoke exploit anywhere in this lab. Each tool gets a definition box at the step that first uses it, listing the arguments this lab hands it:

- three scanners: **zmap**, **masscan** and **nmap**;
- **hydra**, for guessing a password against a login service;
- **curl**, **ncat**, **telnet** and **tcpdump**, for connecting by hand and reading what actually crosses the wire;
- the shell's own **grep**, **awk**, **tee**, **cat**, **head** and **wc**, for cutting scanner output down to the part you need.

Definition: docker exec

Runs one command inside a container that is already running. `-i` keeps your keyboard attached to that command and `-t` gives it a terminal; written together as `-it`, they hand you an interactive shell rather than running one command and exiting. The name after the flags picks the container; the rest of the line is the command to run inside it, here `bash`.

Open a shell on your machine with **docker exec** and stay there; every command in this handout runs on it:

```
# your own terminal: this is the line that opens the shell
docker exec -it 104_L4_LAB_attacker bash
```

Assessment. This lab is graded on eighteen questions, marked inline below as **Question N**, each placed at the step whose output it asks about. Answer all eighteen in a separate document as you work through the steps. Every question is answered from what you see on screen at that step.

Part 1: which addresses hold a host

Definition: Host discovery (ping sweep)

Sending one probe to every address in a range and listing the addresses that answer. The classic probe is an ICMP echo request, the same one `ping` sends, which is why the technique is called a ping sweep. Sweeping `104.1.0.0/24` means 256 probes and one line of output per address that replies.

Definition: zmap

A mass scanner. It sends one kind of probe to every address in a range, as fast as the link takes them, and prints the addresses that answer. One run asks one question. `-M <module>` chooses the probe: `icmp_echoscan` sends ICMP echo requests. The argument after it is the range to sweep, in CIDR form. The answering addresses go to standard output, one per line; the progress lines go to standard error, so the two can be separated.

1. **Sweep the range.** **Host discovery** first: you cannot scan ports on hosts you have not found. Run **zmap** against the range with its ICMP echo module, and let its progress output print rather than hiding it, because you need the rate it reports. `cd` first into `/root/scan`, an empty directory the lab leaves for whatever you produce:

```
# 104_L4_LAB_attacker: your machine, and the only one you type on
cd /root/scan
zmap -M icmp_echoscan 104.1.0.0/24
```

Addresses that answered are printed one per line. The lines beginning with a date are **zmap** logging what it is doing, and none of them carries a rate. The rate is on the progress lines, which begin with an elapsed time such as `0:03`: a running figure first, then the run's average marked **avg**. The last progress line replaces the running figure with **done** and leaves the average.

Question 1. Record the addresses **zmap** reported. State how many addresses it probed, how many answered, and the percentage of the range that answered.

Question 2. Read the average send rate **zmap** printed for this run, in packets per second. The IPv4 unicast space is roughly 3.7 billion addresses once the reserved ranges are taken out. At the rate you measured, calculate how long one sweep of the whole space would take, and give the answer in days. Then give one reason the real figure would be worse than your calculation.

Definition: Redirection, pipes, and the file commands around them

Every command writes to two streams: standard output for its results, standard error for its progress and complaints. `cmd > file` sends standard output to a file, replacing whatever was there; `>>` appends to it instead. `2>` redirects standard error, and `/dev/null` is the sink that discards anything written to it. `cmd1 | cmd2` feeds the standard output of one command into the input of the next, and `cmd | tee file` passes it on while writing a copy to a file. Three commands read a file back: `cat` prints the whole of it, `head -5` prints its first five lines, and `wc -l` counts its lines.

2. **Keep the result.** **Redirect** the addresses to a file so later steps can read them. **zmap** writes its progress to standard error and its results to standard output, so discarding the first leaves the file holding nothing but addresses:

```
# 104_L4_LAB_attacker: in the shell you opened
zmap -M icmp_echoSCAN 104.1.0.0/24 2>/dev/null > ips.txt
cat ips.txt
```

Definition: nmap

A scanner built for asking many questions about a few hosts, which is the opposite trade to **zmap** and **masscan**. The flags this lab uses: `-sn` does host discovery only and scans no ports; `-n` skips reverse DNS lookups; `-PE` restricts discovery to ICMP echo requests; `-Pn` skips discovery altogether and scans every address it is given; `-iL <file>` reads targets from a file; `-p <ports>` sets which ports to scan; `-sV` interrogates each open port to identify the software; `--script <names>` runs scripts against a service. `-oG <file>` writes grepable output, and a bare `-` in place of a filename means standard output, so it can be piped onwards.

Definition: awk

A small language for pulling fields out of lines. A program is a pattern followed by an action in braces, and the action runs on each line the pattern matches. In **awk** `'/Up$/{print $2}'` , the pattern `/Up$/` is a regular expression matching lines that end in Up (`$` anchors a match to the end of a line), and `{print $2}` prints the second whitespace-separated field of those lines. The single quotes stop the shell reading the program as its own.

3. **Sweep the same range with a second tool.** **nmap** does host discovery too, and asks a wider question than one probe type. Run it twice. The first run lets it choose its own probes; the second restricts it to ICMP echo requests with `-PE`, which is the same question **zmap** asked:

```
# 104_L4_LAB_attacker
nmap -sn -n 104.1.0.0/24 -oG -
nmap -sn -n -PE 104.1.0.0/24 -oG -
```

Grepable output is one line per host, designed to be fed to another command rather than read. Chain it into **awk** to pull out just the addresses, in the one-per-line form **zmap** printed them:

```
# 104_L4_LAB_attacker
nmap -sn -n 104.1.0.0/24 -oG - | awk '/Up$/{print $2}'
```

Question 3. The two **nmap** runs do not report the same number of hosts. Give both counts, name the address that appears in one and not the other, and say which of the two runs found it.

4. **Work out why the counts differ.** A ping sweep asks one question: does this address send an ICMP echo reply. RFC 1122 makes answering that a requirement rather than a courtesy: "Every host MUST implement an ICMP Echo server function that receives Echo Requests and sends corresponding Echo Replies." A host that drops echo requests at a firewall is departing from that requirement deliberately, and it stays perfectly reachable for every other purpose while doing so.

nmap's default probe set does not rely on the requirement being honoured. Left to itself it also sends TCP to a couple of common ports, and RFC 9293 requires a TCP endpoint with no listener on the target port to answer with a reset rather than stay silent.

Question 4. Given the two rules above, explain why the output of a ping sweep is a lower bound on the number of live hosts in a range rather than a count of them. Then explain how a TCP probe establishes that a host is alive even when that host answers no ICMP at all.

Definition: masscan

A mass scanner with its own packet engine, covering a range the way **zmap** does but sending TCP SYN's to named ports rather than testing liveness. It prints one line per port that answered. **-p21,23,80** is the port list, comma-separated and attached to the flag. **--rate** caps transmission in packets per second. **--wait** is how many seconds it keeps listening after the last probe goes out, because replies are still arriving once sending has finished; the default is 10, and the countdown appears in the progress line as **waiting N-secs**. Set it too low and slow hosts are missed, which is the same trade every rate has.

5. **Sweep with the third tool.** **masscan** covers a range like **zmap** but tests TCP ports instead of liveness. Give it three ports to try across all 256 addresses, hold it to 1000 packets per second, and let it listen for 3 seconds after the last one:

```
# 104_L4_LAB_attacker
masscan -p21,23,80 104.1.0.0/24 --rate 1000 --wait 3
```

Question 5. You have now swept the same range with three tools. **zmap** and **masscan** between them took seconds; the thorough **nmap** work in Part 2 will take longer per host.

Using your own results, state what **masscan** told you that **zmap** could not, and what neither of them told you that you will need **nmap** for.

Part 2: what each host is running

An address that answers is not yet a target. What makes a host worth attacking is the software listening on it, and that takes a second kind of scan: connect to each port in turn, record which ones accept, then work out what is behind the ones that do.

Definition: Port states: open, closed, filtered

What a scanner concludes from how a port replied. *Open* means something accepted the connection. *Closed* means the host answered but nothing is listening there; a TCP endpoint with no listener replies with a reset, so the host is up and the port is simply empty. *Filtered* is the scanner's verdict when nothing came back at all, which is what a firewall dropping the probe looks like, and also what an address with no host looks like. Open and closed are answers from the port itself; filtered is the absence of one.

6. **Scan the ports of every host you found.** **-iL** makes the list you built in Part 1 the input here, and **-Pn** scans every address on it whether or not it answers a ping, which matters given what you found in Part 1. **tee** shows the output and saves it at the same time:

```
# 104_L4_LAB_attacker
nmap -Pn -n -iL ips.txt -oG - | tee report.txt
```

That scans **nmap**'s default set of 1000 ports per host. Note that **ips.txt** came from the ping sweep, so it is missing a host; add the one you identified in Question 3 by hand before you go on:

```
# 104_L4_LAB_attacker
echo <the address the ping sweep missed> >> ips.txt
nmap -Pn -n -iL ips.txt -oG - | tee report.txt
```

Definition: grep

Prints only the lines of its input that match a pattern, and drops the rest. The pattern comes first, then the file to read, or nothing if the input arrives through a pipe. **'^Host:'** matches lines beginning with **Host:**, because **^** anchors a match to the start of a line; a plain word like **Ports** matches anywhere in the line. The single quotes stop the shell from touching the pattern before **grep** sees it.

To read only the lines that matter, chain the output through **grep** twice, once to keep the per-host lines and once to keep those of them that list ports:

```
# 104_L4_LAB_attacker
grep '^Host:' report.txt | grep Ports
```

Question 6. For every address in your list, record which ports are open. Two of the addresses have none. For those two, quote what **nmap** reports about their 1000 scanned ports, and say what that tells you about whether those hosts are running at all.

Question 7. Apply the **port state** definitions to your own results. On the two addresses with nothing open, every scanned port came back closed and none came back filtered. Explain how that output distinguishes those addresses from an address in the range with no host on it at all, and describe what the same scan would have printed instead if the router in front of them had silently dropped your probes.

Definition: Service banner

The text a service sends when a client first connects, before any request is made. An FTP server greets a new connection with a line beginning 220, and most name their software and version in it. A banner is the cheapest way to identify what is running, because reading it takes no more than opening a connection.

7. **Identify the software, not just the port.** A port number is a convention, not a guarantee: port 80 is registered to HTTP, but nothing stops any program listening there. `-sV` makes `nmap` interrogate each open port and report what actually answered. Run it against each of the three ports you found open, replacing the address and port:

```
# 104_L4_LAB_attacker: run this once per open port you found
nmap -Pn -n -sV -p <port> <address>
```

Question 8. Record what `-sV` reported for all three services. Two of them come back with a software name and an exact version number; the third comes back with a service name and no version. Name which is which, and give a reason a service might not reveal a version to `-sV` when the other two do.

Two of those runs answer in about a second. The third takes tens of seconds and ends by printing a block of escaped bytes that fills most of a screen, starting `SF-Port23-TCP:.` Both follow from the same cause: no signature matched an early reply, so `nmap` worked through the rest of its probe list and then printed the entire exchange.

Definition: Service fingerprint (the SF: block)

What `-sV` prints when a service answered its probes but matched nothing in `nmap`'s signature database. `nmap` sends a fixed series of probes to an open port and compares each reply against thousands of stored signatures. When none matches, it prints the exchange itself, wrapped across continuation lines that each begin `SF:`, and gives a URL for submitting it so that a later release can name the service. Each `%r(Probe,Length,"...")` entry inside the block is one probe by name, the length of the reply to it in hexadecimal, and the reply itself with every unprintable byte escaped, most of them as `\xNN`. Carriage return, line feed, tab and NUL have their own shorter escapes, `\r`, `\n`, `\t` and `\0`.

That block is the **service fingerprint**: `nmap` showing its working, not an error, and nothing in it is a command to run. The probe names say what was sent: `NULL` sends nothing at all and only listens, `GetRequest` sends an HTTP request, `SIPOptions` sends a SIP message. The quoted string beside each one is what this service sent back, so the block is the raw evidence behind the one-line verdict above it, and you can read it.

Definition: Telnet and its option negotiation

Telnet is a remote terminal protocol from 1983, on TCP port 23, carrying everything in the clear. Before either side sends a character of the session, they agree on options such as who echoes typed characters. Each step is three bytes: `\xFF`, the IAC byte, meaning "interpret as command"; then a command, `\xFB WILL` or `\xFD DO` to offer or request an option, `\xFC WON'T` or `\xFE DON'T` to refuse; then the option number, for example `\x01` for echo and `\x03` for suppress-go-ahead. A server sends its opening negotiation as soon as the connection is up, before any prompt.

Every reply in the fingerprint opens with the same run of `\xFF` triples, because that **option negotiation** is the first thing this service says on any connection, whatever the probe that opened it.

Definition: Nmap Scripting Engine (NSE)

A library of scripts `nmap` can run against a service once it knows the service is there. Each script asks one specific question: `http-title` fetches a page and reports its title, `ftp-anon` tries to log in without an account, `banner` simply prints whatever the service says first. They are named after `--script`: one name runs one script, a comma-separated list runs several against the same port in one pass, and a wildcard runs a whole family at once.

8. **Ask service-specific questions.** **NSE** scripts turn "port 80 is open" into a description of the site. Run three of them against the HTTP service at once:

```
# 104_L4_LAB_attacker
nmap -Pn -n -p 80 --script http-title,http-server-header,http-enum <the HTTP
host>
```

Question 9. Record what each of the three scripts returned. For each one, state the single fact it added that you did not already have from the port number and the `-sV` output.

9. **Grab the banners.** The generic `banner` script prints whatever a service sends on connect. Run it against the FTP port and then the telnet port:

```
# 104_L4_LAB_attacker
nmap -Pn -n -p 21 --script banner <the FTP host>
nmap -Pn -n -p 23 --script banner <the telnet host>
```

One of the two returns readable text. The other returns a short run of escaped bytes beginning `\xFF`: the option negotiation and nothing after it, the same triples that open every reply in the fingerprint block. The `banner` script reports what arrives first and stops there, so on this service that is all it has to report.

Question 10. You have two services and only one gave up a useful banner. Explain why the negotiation bytes tell an attacker much less about the telnet service than the FTP greeting told them about the FTP service, and name one thing you could do by hand to learn more about the telnet service than `--script banner` managed.

Definition: Anonymous FTP

A deliberate FTP configuration in which the server keeps an account literally named **anonymous** that anybody may log into, traditionally accepting any string as the password. RFC 1635 limits what that account may do to logging in, listing a restricted set of directories, and retrieving files. It exists so a site can publish files to the public without issuing accounts.

10. **Try the FTP service without an account.** `ftp-anon` attempts an **anonymous FTP** login and reports what it finds:

```
# 104_L4_LAB_attacker
nmap -Pn -n -p 21 --script ftp-anon <the FTP host>
```

Definition: curl

A command-line client that makes one request and prints the response. The scheme at the front of the URL picks the protocol, so `ftp://` makes it an FTP client rather than an HTTP one. `--user <name>:<password>` supplies credentials, and leaving the half after the colon empty sends an empty password, which is what an anonymous login expects. A URL ending in `/` names a directory and `curl` lists it; a URL ending in a filename retrieves that file.

If the login is allowed the script says so, quotes the FTP status code the server returned, and lists what it can see. Then go and read it yourself with **curl**, which `ftp-anon` does not do for you:

```
# 104_L4_LAB_attacker
curl --user "anonymous:" ftp://<the FTP host>/pub/      # list the
    directory
curl --user "anonymous:" ftp://<the FTP host>/pub/<file> # read what is in
    it
```

Question 11. Anonymous FTP is a supported feature, not a bug, and RFC 1635 describes it as the normal way a site publishes files to the public. State what you found in this server's share, and use it to argue whether "anonymous FTP is enabled" is by itself a finding worth reporting. Be specific about what would have to be true for the answer to go the other way.

11. **Check for known vulnerabilities.** `nmap` carries a family of scripts that test for published vulnerabilities in a service. Run the HTTP family, quoting the pattern so the shell hands the `*` to `nmap` instead of expanding it against the files in the current directory:

```
# 104_L4_LAB_attacker
nmap -Pn -n -p 80 --script "http-vuln-*" <the HTTP host>
```

Question 12. Record what this returned. Explain why that result is not evidence that the HTTP service has no vulnerabilities, and state what you would do next with the exact version string `-sV` gave you in Question 8.

Part 3: getting in

Nothing so far has touched a login. The scan produced an account name, a host, and a service that takes a password, which between them are enough to attack a login without any exploit at all.

Definition: Dictionary attack

Guessing a password by working through a prepared list of likely ones rather than every possible combination. The list is what makes it feasible: an eight-character password has too many combinations to enumerate, but if it is one of the few thousand passwords that appear in every breach corpus, a list of a few thousand finds it. Your machine carries a fifty-entry list at `/usr/share/minilabs/passwords.txt`.

12. **Use what the enumeration gave you.** The file you read out of the anonymous FTP share names an account and the host it is used on. Confirm the service on that host is the telnet one you already found open, and look at the password list you are about to use:

```
# 104_L4_LAB_attacker
wc -l /usr/share/minilabs/passwords.txt
head -5 /usr/share/minilabs/passwords.txt
```

Question 13. State the account name and the host address you took from the FTP share. Then explain what this step gave you that no amount of port scanning would have: be specific about which of the two values you now have could not have come from a scan of the telnet host itself.

Definition: hydra

A login guesser. It opens a connection to a service, tries one username and password pair, and moves on to the next. `-l` gives it a single username, where `-L` would take a file of them, and `-P` points at a file of passwords, where `-p` would take a single one. `-t 4` runs four attempts in parallel, and `-f` stops the moment a pair works instead of finishing the list. The target is written `service://host`, and the service half chooses which module `hydra` loads to speak the protocol, here `telnet`.

13. **Guess the password.** Point `hydra` at the account you recovered and the fifty-entry list:

```
# 104_L4_LAB_attacker
hydra -l <the account> -P /usr/share/minilabs/passwords.txt -t 4 -f
telnet://<the telnet host>
```

Dictionary attack done. Run it once more behind `time`, which prints how long the command it is given took, so you can answer the next question:

```
# 104_L4_LAB_attacker
time hydra -l <the account> -P /usr/share/minilabs/passwords.txt -t 4 -f
telnet://<the telnet host>
```

Question 14. Record the password `hydra` recovered and how long the run took. Then calculate: if the service had instead delayed five seconds after every failed attempt, and

hydra still ran four attempts in parallel, roughly how long would the same fifty-entry list have taken? Show the arithmetic.

14. **Log in.** Use the credentials by hand and confirm they work. The `telnet` client takes the host to connect to and nothing else, because port 23 is its default:

```
# 104_L4_LAB_attacker
telnet <the telnet host>
```

There is a file in the account's home directory. Read it, then leave the session open for the next step or log back in when you need it.

Question 15. Try one login with the correct account name and a wrong password, and watch what the server does after it refuses you. Explain how that behaviour affects an attacker guessing passwords, and contrast it with what a lockout after three failures would have cost them.

Definition: tcpdump

Prints the packets crossing an interface as they arrive. `-i <interface>` picks the interface to listen on, `-n` leaves addresses and ports as numbers instead of looking up names, and `-A` prints each packet's payload as ASCII text underneath its header line. The quoted expression at the end is a filter, and only packets matching it are shown: `tcp port 23` matches TCP segments with 23 as either the source or the destination port, so both directions of the session appear. The quotes keep the shell from splitting the expression into separate arguments.

15. **Watch the credentials cross the wire.** Telnet applies no encryption to anything it carries, so every byte of that session crossed the link exactly as it was typed. Open a second terminal on your own machine and start a `tcpdump` capture, then log in again in the first:

```
# 104_L4_LAB_attacker: a SECOND terminal on your machine
docker exec -it 104_L4_LAB_attacker bash
tcpdump -n -i 104-ext -A 'tcp port 23'
```

```
# 104_L4_LAB_attacker: back in your first terminal
telnet <the telnet host>
```

Question 16. Find the account name and the password in the capture and quote the lines they appear on. Explain what an attacker sitting on the router between the two hosts, who had captured this session without doing any scanning at all, would have obtained.

Question 17. Three separate weaknesses had to line up for this to work: the protocol carrying the login, the password itself, and how the service treats failed attempts. Name the specific weakness in each case, then say which single one you would fix first and justify the choice in terms of what an attacker still could and could not do afterwards.

Question 18. Replacing telnet with SSH is the change usually recommended first. Of the three weaknesses you named in Question 17, state which ones switching to SSH would fix and which would survive unchanged, and explain why.

Checks: what “done” means

- **Part 1:** you have a file of addresses that includes the host the ping sweep missed, and you can say which probe found it.
- **Part 2:** you have, for every address, either a list of open ports or a statement that it has none; and for each open port, the software and version behind it.
- **Part 3:** you can log in over telnet with credentials you recovered rather than were given, and you have seen both of them in a packet capture.

`scripts/status.sh` confirms every service in the lab is up and reachable, so you can tell a broken lab from an empty scan result. It does not list the addresses or ports, because those are what you are being asked to find. `scripts/reset.sh` deletes your scan output and restarts the services, so a part can be run again from nothing.

Safety. Everything here lives on one isolated router and switch with no route off the host. Every probe, guess and capture only ever reaches the lab’s own containers. Running these same tools against a network you do not own or have written permission to test is an offence in most jurisdictions, including under the Crimes Act 1961 in New Zealand, and it is a breach of the acceptable-use policy of every university and consumer internet provider.

When you are done, tear the lab down. Choose **Teardown** in the TUI, or run `scripts/teardown.sh` directly, to remove the containers and their wiring so nothing is left running on your host. **Reset to baseline** only clears your scan output; only **Teardown** deletes the lab.